

Control Language Multifunction Controller Data Entry

PC11-585

**Implementation
Hiway Gateway - 2**

Control Language Multifunction Controller Data Entry

**PC11-585
Release 500
12/97**

Copyright, Trademarks, and Notices

© Copyright 1995 - 1997 by Honeywell Inc.

Revision 02 - December 16, 1997

While this information is presented in good faith and believed to be accurate, Honeywell disclaims the implied warranties of merchantability and fitness for a particular purpose and makes no express warranties except as may be stated in its written agreement with and for its customer.

In no event is Honeywell liable to anyone for any indirect, special or consequential damages. The information and specifications in this document are subject to change without notice.

TotalPlant and TDC 3000 are U. S. registered trademarks of Honeywell Inc.

Other brand or product names are trademarks of their respective owners.

About This Publication

This publication provides instructions for the following tasks:

- Creating CL source files, and entering and editing data in the source files
- Using the Engineering Personality's Utilities to manage and manipulate files related to CL
- Using the CL Compiler commands and options that are part of the Command Processor activity on the Engineering Main Menu

This publication supports **TotalPlant** Solution (TPS) system network Release 500. TPS is the evolution of TDC 3000.

Change bars are used to indicate paragraphs, tables, or illustrations containing changes that have been made to this manual effective with this update. Pages revised only to correct minor typographical errors contain no change bars.

Table of Contents

1 INTRODUCTION

- 1.1 Who This Publication is Intended For
- 1.2 How to Use this Publication
- 1.3 Related Publications

2 PREREQUISITES FOR A CL DATA ENTRY SESSION

- 2.1 Preceding System Startup Tasks Completed
- 2.2 User Volumes Established with Sufficient Capacity
- 2.3 Engineering Personality Running and Needed Volumes Available
 - 2.3.1 Universal Station and Volume Handling Guidelines
- 2.4 Prerequisites for a CL/MC Compile and Load Session

3 TYPICAL CL DATA ENTRY SESSIONS

- 3.1 How to Use this Section
- 3.2 Installing a CL/MC Sequence Program
 - 3.2.1 HG Message Libraries
 - 3.2.2 Prepare to Load the .MO Object File
- 3.3 Moving CL Applications Between Systems

4 ENGINEER'S KEYBOARD

- 4.1 CL Data Entry Key Functions

5 COMMANDS

- 5.1 Using Utility Commands in CL Data Entry
- 5.2 Using Text Editor Commands in CL Data Entry
 - 5.2.1 How to Create a CL Source (.CL) File
 - 5.2.2 How to Copy a File, Edit it, and Store it in a New File
- 5.3 CL Compiler Commands
 - 5.3.1 Compiler Command Descriptions
 - 5.3.2 How to Invoke Compiler Commands
- 5.4 How to Use Command Files

6 ERROR INDICATIONS AND RECOVERY

- 6.1 How Errors are Indicated
 - 6.1.1 CL Data Entry Error Messages
- 6.2 Correcting CL Source File Errors
- 6.3 How to Determine the Location of a CL/MC Runtime Error

INDEX

INTRODUCTION

Section 1

This section provides

- A description of the intended use for this publication.
- References to other publications needed or useful for compiling, linking, and loading CL packages.

1.1 WHO THIS PUBLICATION IS INTENDED FOR

This **publication** is prepared for use by people such as process engineers, control-system engineers, and application engineers who need to install custom functions in a **TotalPlant** Solution (TPS) system, using CL/MC, the Control Language for the Multifunction Controller and the Advanced Multifunction Controller.

1.2 HOW TO USE THIS PUBLICATION

Use this publication when

- You need to determine what work must be accomplished before you can install or modify a CL program. Section 2 should be especially useful for this purpose.
- You are ready to install a new CL program or to change an existing CL program.

The expected use of Sections 2 through 5 of this publication is as follows:

- **Section 2 - Prerequisites**—Use this section to determine what must be available to install or to change CL programs and what startup tasks must be completed.
- **Section 3 - Typical CL Data Entry Sessions**—Use this section to review the major steps to install or change a CL program and the recommended order of those steps.
- **Section 4 - Engineer's Keyboard**—Use this section when you don't know or are not sure of the effect of pressing a key on the Engineer's Keyboard.
- **Section 5 - Commands**—Use this section to determine the detailed steps in executing the commands used to install or to change CL programs, and for a description of the functions of the CL compiler and linker.
- **Section 6 - Error Indications and Recovery**—This section provides guidance in recovering from human errors.
- **Index**—An alphabetized list of CL data entry topics with references to paragraph numbers in Sections 1 through 6.

Before using the procedures in Sections 3 through 5 of this publication, you should review the publications mentioned under 1.3.

1.3 RELATED PUBLICATIONS

The following are publications that contain reference information related to CL/MC data entry, or provide procedures that are related to CL/MC data entry.

- *Control Language/Multifunction Controller Reference Manual, PC27-510*, in the *Implementation/Hiway Gateway* - 2 binder. This manual defines the Control Language, including its environment. The language's elements, syntax, and rules are defined. Also defined are the relationships of the parts of CL programs and their sizes and limitations.
- *System Control Functions, SW09-501*, in the *Implementation/Startup & Reconfiguration* - 2 binder. This manual covers data acquisition and control functions that are independent of the nodes that participate in the functions.
- *Hiway Gateway Control Functions, HG09-501*, in the *Implementation/Hiway Gateway* - 1 binder. This manual covers data acquisition and control functions specific to the Hiway Gateway.
- *Multifunction Controller Algorithms Engineering Data, BC-10-01*, in the bookset for hiway-based products, in the *BASIC Product Manual*. Describes the functions of all algorithms available in Multifunction Controllers.
- *Multifunction Controller Specification and Technical Data, BC-03-01* (or *Advanced-Multifunction Controller Specification and Technical Data, BC-03-05*) in the bookset for hiway-based products, in the *BASIC Product Manual*. Describes the functions of the Multifunction Controller, other than algorithms.
- *Engineer's Reference Manual, SW09-505*, in the *Implementation/Startup & Reconfiguration* - 2 binder. Provides guidance and tips that are useful in designing and implementing the system configuration, and in starting up the TDC 3000 System.
- *Hiway Gateway Parameter Reference Dictionary, HG09-540*, in the *Implementation/Hiway Gateway* - 1 binder. This publication provides information about all standard parameters, including parameter names, data types, value ranges, and access levels.
- *System Startup Guide/Cartridge Drive, SW11-504*, in the *Implementation/Startup & Reconfiguration* - 1 binder. We recommend that you use this guide the first time you use any of the activities on the Engineering Main Menu. This guide will provide step-by-step procedures for all available configuration activities and will define 31 system startup tasks. Task 28 is Build, Compile, and Load CL Programs.
- *Data Entity Builder Manual, SW11-511*, in the *Implementation/Engineering Operations* - 1 binder. This publication describes the use of the Data Entity Builder (DEB). The **DEB is used** for eight of the activities on the Engineering Main Menu. Of those eight, data-point building is the most important to CL data entry, because it is used to build the data points that CL programs work with, and to attach Custom Data Segments to data points.

- *Command Processor Operation, SW11-507*, in the *Implementation/Startup & Reconfiguration - 1* binder. This publication provides instructions for the use of all of the functions available through the Utilities pick on the Engineering Main Menu. Utilities are used to set up default volume pathnames, to manage volumes and files, including those used by CL, to catalog and list the content of volumes and files, and to display and print the content of CL files.
- *Text Editor Operation, SW11-506*, in the *Implementation/Engineering Operations - 3* binder. This publication defines how to use the Text Editor. It is used to create CL source files on user volumes, and to enter and edit information in those files.
- *Process Operations Manual, SW11-501*, in the *Operation/Process Operanons* binder. This manual provides instructions for use of the Universal Station's Operator personality. CL/MC (sequence) programs are loaded in the MC through this personality.

PREREQUISITES FOR A CL DATA ENTRY SESSION

Section 2

Use this section to determine what must be available to install or to change CL programs, and what startup tasks must be completed.

2.1 PRECEDING SYSTEM STARTUP TASKS COMPLETED

You can create CL source files, enter data in them, and edit them at any time. CL/MC programs can be compiled any time after System Startup Task 21 is completed, and can be loaded any time after Tasks 22, 25, and 30 are completed (Task 22 must be complete only if logic blocks are referenced by a CL/MC program). See the tasks shown in the *System Startup Guide* for further information.

CL/MC data entry consists of creating a source file, compiling source files, and loading CL programs into MCs, where they will be executed.

The Link command does not apply to CL/MC sequence programs. A sequence program (the .MO object file, which must be on Volume &Enn—refer to heading 3.2.2) is loaded into a Process Module (a sequence slot in the MC) from the Process Module Detail display in the Universal Station's Process Operator personality. To load a CL/MC program, at least one US must be running the Operator Personality, so System Startup Task 30 must be completed.

2.2 USER VOLUMES ESTABLISHED WITH SUFFICIENT CAPACITY

You will store your CL source files on user volume floppies or in user volumes on an HM. The CL Compiler creates listing files, error-listing files, and object files on the volume that contains the source file (Table 2-1 lists all of the volumes used in CL data entry). You will find it easier to use the HM.

Whether you use removable media (floppy diskettes or cartridge disks) or the HM, you must configure one or more user volumes with sufficient capacity to store all of your source files and Compiler-generated files. Unless you create an unusually large number of CL programs, the CL user volumes will be much smaller than those that contain IDFs with DEB-built entities. Guidelines for estimating the size of CL-user volumes are provided under 7.2 in the *Engineer's Reference Manual*.

You can use Task 4 (removable media) or Task 10 (HM) to configure your user volumes (see heading 3.2.2). User volumes should be configured in Task 10 during the initial startup, but the HM volume configuration can be changed to include user volumes by using the on-line reconfiguration procedures (see *Network Data Entry* in the *Implementation/Startup & Reconfiguration - 1* binder). For a more detailed explanation of floppy formatting, refer to *Command Processor Operation* in the *Implementation/Engineering Operations - 1* binder.

We recommend that you periodically copy any CL-user files you have on an HM to removable media, so that you can recover them should something cause the data on the HM to be lost. Use the Copy command to do this.

2.3 ENGINEERING PERSONALITY RUNNING AND NEEDED VOLUMES AVAILABLE

All of the activities described in this publication take place at a Universal Station with the Engineering Personality running. To begin any of these activities, the Engineering Main Menu must be on the screen. To load the Engineering Personality and to get the Main Menu on the screen, you can use Startup Task 2 or refer to the *Configuration Data Collection Guide*.

Table 2-1 lists all of the volumes and files involved in installing and changing CL programs and Custom Data Segments. These volumes should be available on an HM or on removable media, as needed. It is easier to have them on an HM, so mounting and dismounting disks isn't necessary.

Table 2-1 — Volumes and Files Used in Installing and Changing CL/MC Programs

Volume	File Name and Suffix	Use
&OV1	(Several)	Engineering Overlay 1—contains the Text Editor, Utilities, and Data Entity Builder (DEB)
&OV2	(Several)	Engineering Overlay 2—contains the CL Compiler
&ASY	(Several)	Contains the Standard Parameter Lists
&HGG	(Several)	Contains the HG GDFs.
&Enn	(see 3.9)	Contains CL/MC object (.MO) files
CL	(filename).CL	CL source file (created by the user)
"	(hhbbsiii).MO	CL/MC object file (created by the Compiler)
"	(filename).LS	CL listing file (created by the Compiler)
"	(filename).LE	CL error listing file (created by the Compiler)
Note: CL is a user volume that can have any user-defined name. CL is the default name that appears on the Modify Default Volume Path Names display. You can change the name on that display to the name of your user volume(s).		

2.3.1 Universal Station and Volume Handling Guidelines

CL data entry requires a Universal Station that has an Engineer's Keyboard. It is useful to have a printer connected to the US so that you can make printed records of your work. If you are using removable media, the US should have two drives connected to it, so that the transfers will be faster than if they have to go over the LCN.

Execution of any command that can change the system database or the process database requires the key switch on the Operator Keyboard to be in the ENGR position. It is simpler to check the switch when you start a session and leave it in ENGR until you are finished.

It is very helpful to have another Universal Station in the same console with the Operator Personality running. You can then look at the detail displays for the points associated with your CL programs, to verify that parameter values mentioned in this publication are as described, and to see the effects of your work.

It is better to have one or more user volumes for your CL files (see Table 2-1) on an HM (Startup Task 10) and to have the complete software complement on an HM (Startup Task 16). If you use removable media, use one user volume for all CL files, and mount and dismount disks, as needed. For example,

- When using the Text Editor or Utilities, &OV1 should be in one drive and your user volume in the other.
- When you enter CL from the COMMAND PROCESSOR to use the CL Compiler, &OV2 should be in one drive and your user volume in the other. In some situations, you need to replace &OV1 with &ASY after the Control Language Compiler/Linker display appears. The procedures in Sections 3 and 5 tell you when.

Before you begin any activity on the Engineering Main Menu, you should select SUPPORT UTILITIES and check the Modify Default Volume Path Name display to see that the correct sources (\$Fn or NET) and the correct volume names are entered. If not, change and then press ENTER.

2.4 PREREQUISITES FOR A CL/MC COMPILE AND LOAD SESSION

Before CL/MC programs can be compiled the following must be true:

- A. The **Data Hiway** must be configured (System Startup Task 19).
- B. The **MC box/slot entity** must be built and loaded (System Startup Task 20). This entity must contain correct values in the following parameters (see *Network Form Instructions*); BOXCLINK (box C-link number), CLINKNUM (C-link address), and LIBRYNUM (message-library number).
- C. The **HG Libraries entity** must be built and loaded (System Startup Task 25). It can contain blanks (Compiler automatically enters words), or message words can be built into it. If the compilation of the CL/MC program will add words to it, there must be enough space left in this entity.
- D. The **Process Module Data Point**, to which the sequence will be loaded, must be built and loaded in the HG. Any data points, including any other Process Module data points that the sequence references, must also be built and loaded.

Before CL/MC programs can be loaded, the following must be true:

- A. The MC that will receive the CL/MC program must be loaded and running. Also, any MCs that contain Process Module Data Points that are referred to by the CL/MC program should be loaded and running to prevent runtime errors when attempting to access the point.
- B. There must be sufficient sequence memory in the MC to accept the CL/MC program (see 3.2.2).
- C. The preparations described under 3.2.2 must be completed; this includes ensuring that the .MO (object file) is in volume &Enn, where nn = hiway number the MC is on.

TYPICAL CL DATA ENTRY SESSIONS

Section 3

3.1 HOW TO USE THIS SECTION

Typical CL data entry sessions are described in this section for these two reasons:

- To provide an overview of typical sessions for new users.
- To define the order in which we recommend you accomplish the major steps.

References to detailed instructions for these major steps are provided.

3.2 INSTALLING A CL/MC SEQUENCE PROGRAM

Use the following major steps to install a CL/MC sequence program or to change one.

1. From the Engineering Main Menu, select SUPPORT UTILITIES and then select MODIFY VOLUME PATHS. On the Modify Default Volume Path Names Display check that the pathnames for the following volumes are correct (see Table 2-1):

CL SOURCE/OBJ

examples; \$F2>CL> or NET>CL>

CL OVERLAY

examples; \$F1>&OV2>
or NET>&OV2>

NETWORK CONFIG

examples; \$F2>&ASY>
or NET>&ASY>

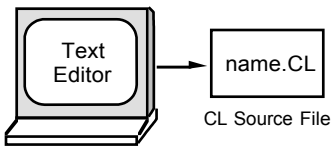
2981

If necessary, correct the pathnames and press ENTER. The display reappears with all pathnames in blue.

NOTE

If you are using removable media (floppy diskette or cartridge disk), just before you enter CL from the COMMAND PROCESSOR display, mount &OV2 in the drive indicated by its pathname. When the Compiler/Linker display appears, replace &OV2 with &ASY and leave it there for the remainder of this session.

2. Use the Text Editor to create the source file (see 5.2.1) on the CL Source/Object Volume named in step 1.



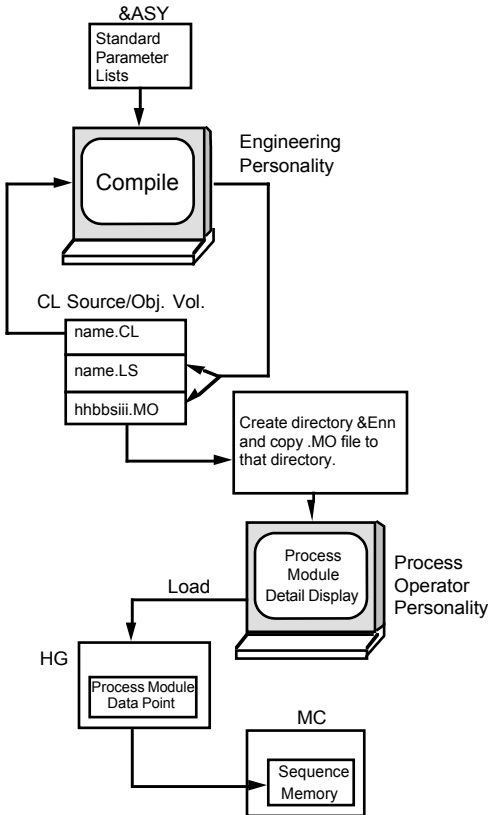
1348

3. Compile the CL source file (see 5.3.2). If you use the -UL option, sequence names, phase names, step names, and SEND string items are automatically entered in the HG Library.

If no errors are detected, these files are created on the CL source/object volume:

(filename).LS

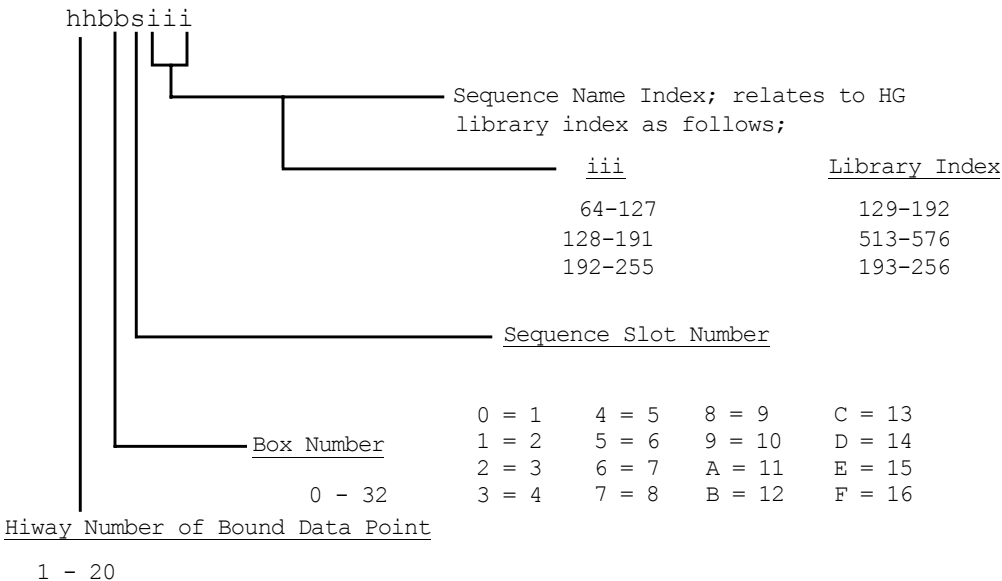
(hhbbsiii).MO



1362

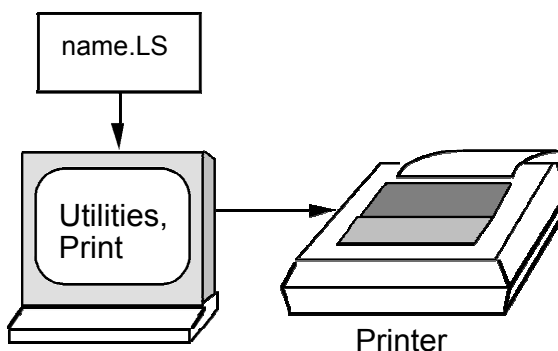
Where (filename) is the name of

is the object file name, which is in the following form:



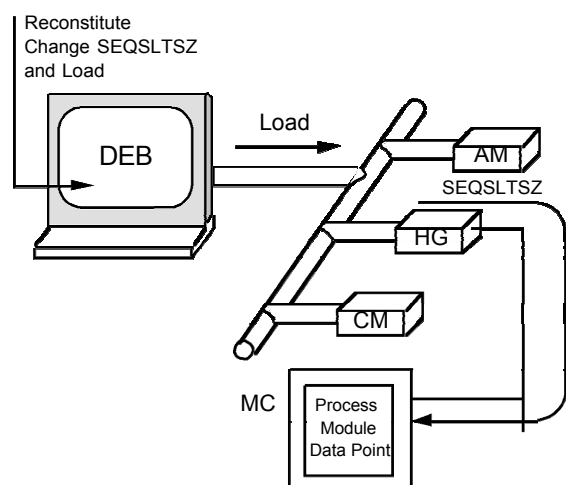
3087

4. Use the Print command (see 5.1) to look at the listing (.LS) file. Find and write down the object size information from the listing. The object size is stated as blocks of 16 words.



1363

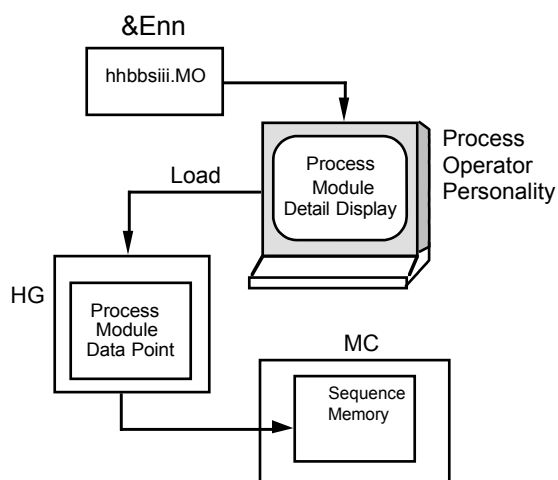
5. Use the DEB to reconstitute the Process Module point for this sequence (see the *Data Entity Builder Manual*). Make sure the value in parameter SEQSLTSZ allocates enough sequence memory space to accommodate the object size determined in step 4. It's a good idea to make it large enough to accept future compilations as well.
6. When the PED-status line indicates no errors, no missing data, and no unENTERed data, write the Process Module point to an IDF and load it in its node.



1364

7. Prepare to load the .MO object file according to 3.2.2 (the .MO file must be in volume &Enn), then

Process Module display in the USS' On Process Personality.



1365

3.2.1 HG Message Libraries

Each Hiway Gateway has four message libraries. Each MC is assigned to one of these libraries by an entry in parameter LIBRYNUM in the MCs Box/Slot entity (Startup Task 20).

If SOPL programs are to be run in an MC, the SOPL message library should be exactly copied to the appropriate HG message library. This provides correct messages, sequence names, and step names on the US(s). CL/MC compilations can add entries to the library, if necessary, but cannot delete any entries in the library.

If no SOPL programs need to be considered, we recommend that you first use the DEB to initialize the libraries with blanks (Startup Task 25). You can do this by selecting BUILD HG LIBRARIES on the Engineering Main Menu, keying in the library number in parameter LIBRYNUM (corresponds to the MC Box/Slot entity whose LIBRYNUM parameter has the same number), and pressing ENTER to select the default values for the messages, which are all blanks. Then load the HG Library entity in its HG.

These are the two methods you can use to add message items to an HG message library:

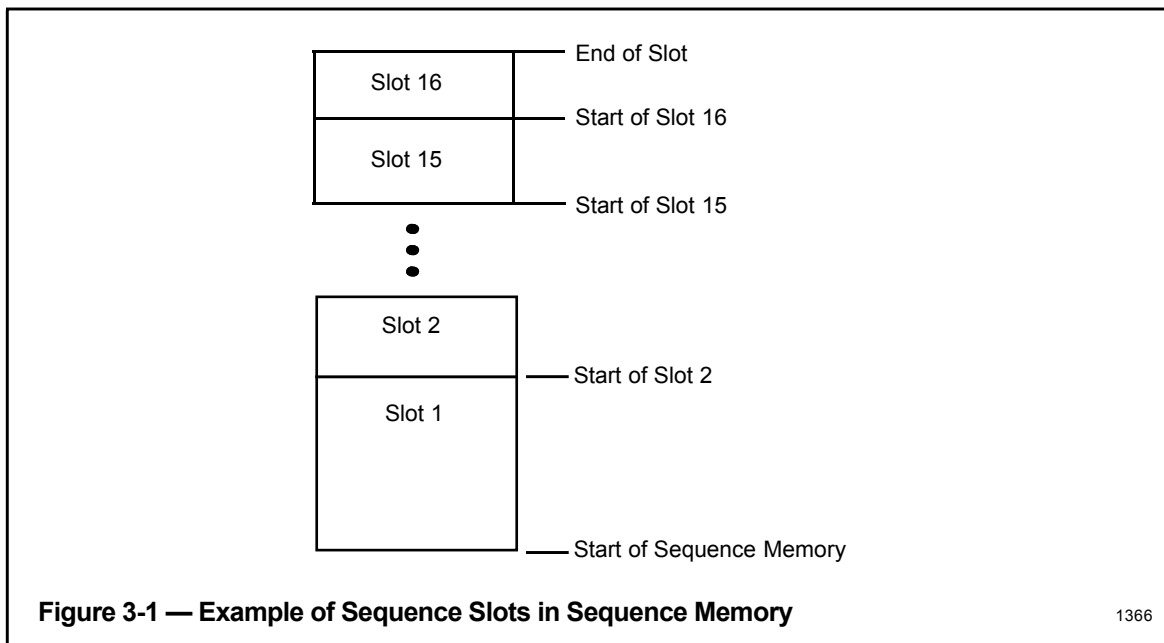
- Let the CL Compiler automatically do it. This is permitted by using the Compiler's -UL option.
- Use the DEB to key message words into the HG Libraries entity. You can reconstitute, key in new words or change words, and then reload or you can build a fresh version and load, using the overwrite option if another version already exists in the HG. The CL Compiler uses words entered through the DEB when generating code for SEND statements. If you compile without the -UL option, a .LE file is generated that lists needed words that were not found in the HG message library.

Once a compilation that uses an HG message library has been run, it is best not to change any entries in that library. This could cause incorrect messages and the display of wrong step, phase, and sequence names. If you have to make any changes to an HG library, you should repeat all compilations that use the library to ensure that the messages, step names, phase names, and sequence names are correct.

3.2.2 Prepare to Load the .MO Object File

The CL/MC object file(s) (.MO) (and any upline-loaded SOPL object-files) must be available on a volume whose name is &Enn, where nn is the hiway number. Here are two ways to make such a volume available:

- Use the Engineering Personality's Command Processor's Create Directory command to create the directory &Enn on the user volume that has your .MO file, on another user volume, or on a disk/diskette.
- In System Startup Task 10, configure a "CL/MC Data Hiway" volume for each hiway needing a download volume.



1366

Use the Engineering Personality's Command Processor's Copy or Move command to copy/move your .MO file to the &Enn volume or directory. This volume/directory could be used to store your tested (debugged) CL/MC programs (and SOPL sequences), keeping them separate from untested CL/MC programs on your user volume.

Sequence programs are loaded, one after the other, into the MC's sequence memory, as shown on Figure 3-1. If you reload a changed sequence whose size is different than it was, you may not be able to load it, or you may get into trouble when you try to reload subsequent slots. Here are some guidelines:

- The size of each sequence slot is stored in parameter SEQSLTSZ in the Process Module data point. The total of all the sequence-slot sizes in the MC can't exceed the available sequence memory. If a sequence program is larger than its configured slot size, it cannot be loaded.
- The location in memory where a sequence begins is determined by the sum of all of the preceding slot sizes.
- Changing the size of a sequence slot changes the starting location of the next slot, the next time that slot is loaded (slot 1 is first, 16 is last).
- When the size of a sequence slot is changed, subsequent reloading of a sequence program to that slot may overwrite the sequences in slots before or after it. This causes a loading error, unless the slots on that sequence are in the Off state or the MC is idle. If an overwritten sequence program starts to execute before it is itself reloaded, a checksum error occurs and execution stops.
- Changing the MC box state to "Processing" initializes slot sizes to the configured sizes.

If you change the size of a sequence slot, you must consider the size of all of the other slots. When a changed sequence program with a different slot size is loaded, all subsequent sequence slots should be reloaded.

3.3 MOVING CL APPLICATIONS BETWEEN SYSTEMS

CL structures can be moved from one LCN-based TDC 3000 System to another, only in CL source file form. Object files, IDFs, and GDFs contain internal representations of names that apply to only the system they were created on. Specifically,

- **Data points** in the destination system must be built using the **DEB's Exception Build** command.
- **CL/MC source files** must be recompiled on the destination system, with the associated Process Module data points and HG Message-Library entities (see 3.2.1) loaded in the HG(s) in the destination system.

It's likely that before you do any of these activities you will need to edit the CL-source files on the destination system, to change point names and to tailor the source file for the destination system.

ENGINEER'S KEYBOARD

Section 4

Use this section when you don't know or are not sure of the effect of pressing a key on the Engineer's Keyboard.

4.1 CL DATA ENTRY KEY FUNCTIONS

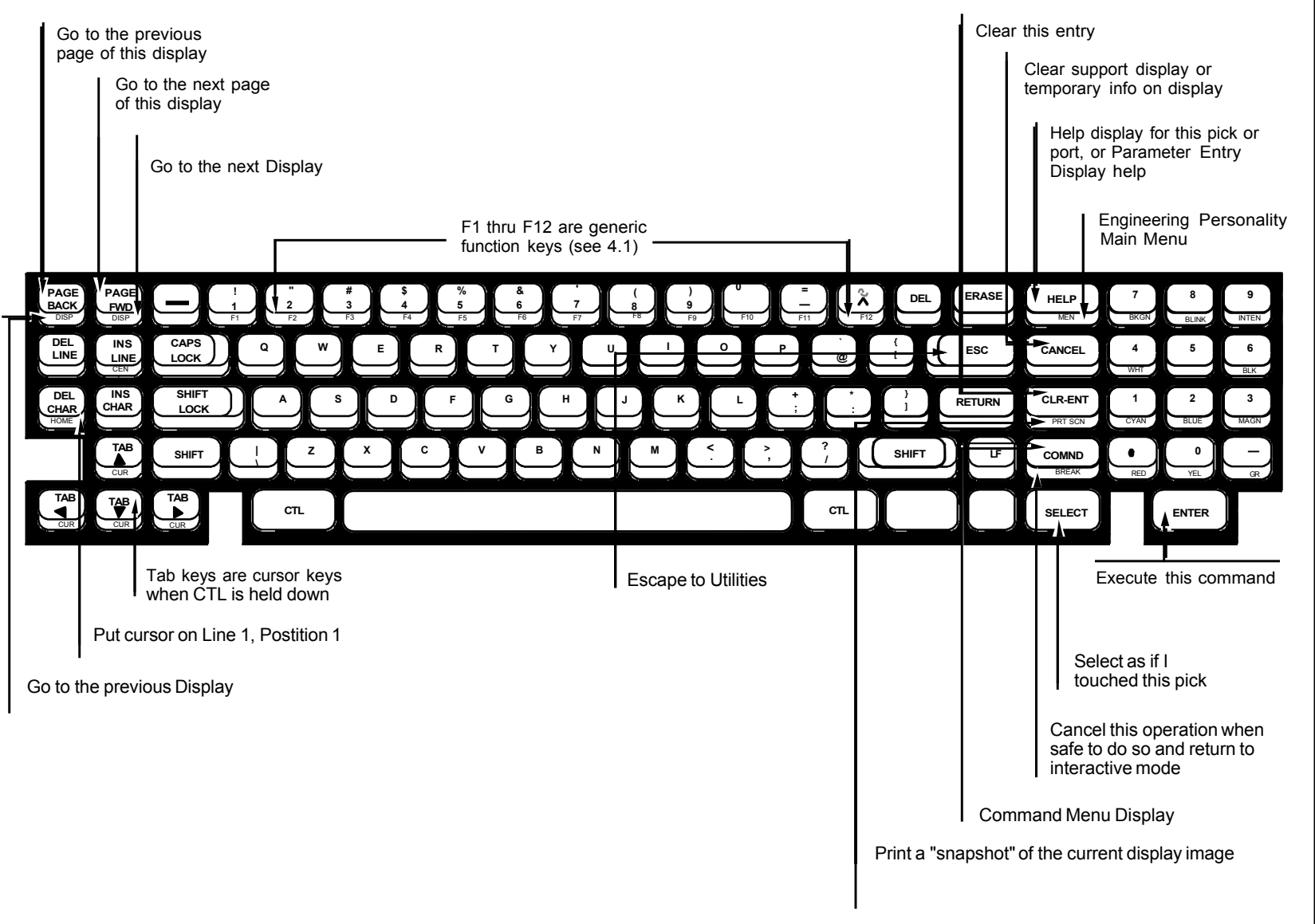
The Engineer's Keyboard is required for interaction with the Engineering Personality's Utilities, Text Editor, and CL Programs activities. Most of the keys function as you would expect. Figure 4-1 clarifies the functions of several of the keys.

Several keys have dual functions and dual markings—symbols on the top of the key and other symbols on the front edge of the key. The secondary function, indicated on the front edge of these keys, is obtained when one of the two CTL keys is held down while the function key is pressed.

The generic-function keys, F1 through F12, are used to execute Text Editor commands, as described in Section 3 of the *Text Editor Operation* manual. These keys are not used in the Utilities and CL Programs activities.

The color and behavior keys (BKGND, BLINK, INTEN, WHT, BLK, CYAN, BLUE, MAGN, RED, YEL, GRN) don't operate with the Utilities, Text Editor, and CL Programs activities.

Figure 4-1 — Engineer's Keyboard Special Key Functions



1194

COMMANDS Section 5

This section provides

- An overview of Text Editor and Utilities commands that are related to CL-data entry (see 5.1 and 5.2).
- Detailed descriptions of the CL Compile and Link commands (see 5.3).

Section 3 describes how these commands are used to install and to modify CL programs. Section 6 provides guidelines for recovering from errors indicated as the Compile and Link commands are executed.

5.1 USING UTILITY COMMANDS IN CL DATA ENTRY

The default-volume pathnames and utilities are used in CL data entry. The Modify Default Volume Pathnames display is called up by selecting the `SUPPORT UTILITIES` pick on the Engineering Main Menu and then selecting `MODIFY VOLUME PATHS` on the Support Utility Menu. (Alternately, you can select the `COMMAND PROCESSOR` pick on the Engineering Main Menu, and then type `SP` and press `ENTER`). The utilities are called up by selecting the `COMMAND PROCESSOR` pick on the Engineering Main Menu.

The following are the functions that you will probably use most in CL data entry (command-file utilities Execute Command, Prompt Out, Pause, and End are described under 5.4):

- **Modify Default Volume Pathnames**—This function is used to set up the pathnames to the files that are created and used in CL data entry. Examples of such pathnames are provided in this publication at heading 3.2—Install a CL/MC Sequence Program.

Instructions for calling up the Modify Volume Pathnames Display are under 4.3 in the *Command Processor Operation* manual. Instructions for changing the default pathnames are in Section 6 of that publication.

- **Copy**—Used to copy a file or files from one volume to another, or on the same volume. We recommend that you copy all of your files to removable media, in case a file is lost or damaged. The Copy and Copy Volume commands are described under 5.6 and 5.7 in the *Command Processor Operation* manual.
- **Create**—Used to name a volume on a floppy diskette or cartridge disk and to initialize the diskette/disk so that it can be used on the TDC 3000 System. This command is used to create user volumes that you use in CL data entry, or on which you make backup copies of HM files. This command is described under 5.8 in the *Command Processor Operation* manual.
- **Create Directory**—Used to create a directory under an existing volume. An example is described in subsection 3.2.2 in this publication, Prepare to Load the MO Object File.

- **List Names**—Used to list the attributes of a volume and its files. You can use this command to determine if specific files are in a volume, the time and date they were created, their size, and other information. For your records, you might want to use the Data Out command to direct the list to a printer as well as to the screen. The List Names command is described under 5.13 in the *Command Processor Operation* manual.
- **Data Out**—Directs the output generated by a subsequent Print Command to a printer or file as well as to the display screen, or to only the screen. This command is executed directly from the COMMAND PROCESSOR. To direct the output to go to the screen and a printer, on the Control Language Compiler/Linker display, key in a command as in this example:

```
DO $Pn
```

and press ENTER. In this command, n is the printer number (which is available on the Console Status Display). To direct that the output is to go to only the screen (this is the default situation), key in DO and press ENTER.

- **Print**—Used to display or to print the content of a file. Use this command to look at your listing (.LS) and error (.LE) files. This command is executed directly from the COMMAND PROCESSOR. Key in a command as in this example:

```
PR NET>CL>VLR_CALC.LE
```

and press ENTER. When the whole content of the requested file has been displayed, "Print Complete" appears at the bottom of the display area. If a Data Out command has also directed the output to a printer, the content of the file is simultaneously displayed and printed.

- **Delete**—Deletes a file from a volume. If you make changes to CL source files, you may want to copy the original file to a file with a slightly different name, make the changes in the new file, install those changes, test them, and when the modified structure is operating properly, delete the original file. For example, you might name your original file CALC1 and the new file CALC2. When CALC2 is compiled, installed, and successfully running, you could delete CALC1. The delete command is described under 5.10 in the *Command Processor Operation* manual.

5.2 USING TEXT EDITOR COMMANDS IN CL DATA ENTRY

The following procedures describe the Text Editor functions you will probably use most frequently in CL data entry. For more detailed descriptions of Text Editor functions, refer to the *Text Editor Operation* manual.

5.2.1 How to Create a CL Source (.CL) File

1. You access the Text Editor through the COMMAND PROCESSOR pick on the Engineering Main Menu display.
2. Key in ED followed by the pathname for the file to be created. Be sure to include the .CL suffix. For example:

```
ED NET>CL>CALC1.CL
```

The file name cannot begin with "\$."

3. Press ENTER. A blank Text Editor display appears, ready for you to key in your source file information.
4. Key in your source file. The form and content of source files for all CL structures are in the *Control Language/Multifunction Controller Reference Manual*.
5. After you have keyed in and visually checked your entire source file, end the Text Editor session by pressing LF, then E, and RETURN (or CTL F1 followed by CTL F2). The source file is transferred from the Text Editor's temporary file to the user volume that you named in step 2, and your source (.CL) file is now ready to compile.

5.2.2 How to Copy a File, Edit it, and Store it in a New File

Often, you will use several similar CL structures and you can save time by copying a source file, editing the copy, and storing the result in a new file.

1. You access the Text Editor through the COMMAND PROCESSOR pick on the Engineering Main Menu display.
2. Key in ED followed by the pathname for the file to be copied. Be sure to include the .CL suffix. For example:

```
ED NET>CL>CALC1.CL
```

3. Press ENTER. The first 23 lines of the file are displayed.

4. Edit the copy of the file that is now on the Text Editor display to make the changes that are needed in the new file. When you finish, leave the cursor on the first character you want in the new file.
5. Hold CTL and press F7. Four new prompters appear (F1=Define, F2=Get, F3=Put, and F4=UnProt).
6. Hold CTL and press F1. A prompter appears that asks for the pathname for the new file.
7. Key in the pathname of the new file and press ENTER. Here is an example of the pathname:

CALC2.CL

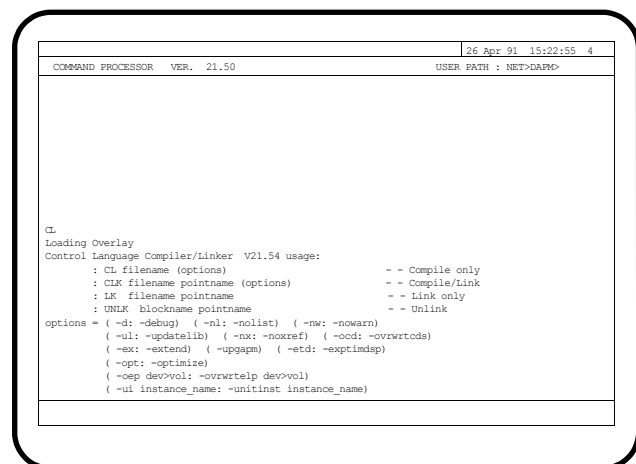
8. Hold CTL and press F3. A prompter appears that asks how many lines to output. If you want to store the whole file, key in 9999 (this is more lines than the largest file). Otherwise, key in the number of lines you want to store in the new file.
9. Press ENTER. You now have two copies of the file, the original one that you copied and the new one that you named in step 7. In this example, they are both in the same volume. If a different volume is needed, precede the filename entry in step 7 with the entire pathname.

5.3 CL COMPILER COMMANDS

The CL Compiler commands are available through the COMMAND PROCESSOR pick on the Engineering Main Menu. The Compiler reads CL/MC source (.CL) files and creates object files from them. Object files are files that contain the CL structures in the form that is executed by the destination node. The object files subsequently are loaded to the Multifunction Controller as explained at heading 3.2.2.

5.3.1 Compiler Command Descriptions

This Control Language Compiler/Linker Display appears when you type CL on the COMMAND PROCESSOR display and press ENTER. When the display first appears, the cursor is in the command-entry/prompter region, as shown here. The area between the command-entry/prompter region and the title line displays the results of command execution.



2989

5.3.1.1 Commands

For CL/MC, only the Compile command can be executed from this display. Execution is requested by keying in the command line with any options and pressing ENTER.

- **Compile**—Compiles CL/MC source files, creating object and listing files, or if errors are detected, an error file. Form:

```
CL (filename) -(options)
```

In the command form above, "(filename)" is the name of the CL source file. The source file must have a .CL suffix, but use of the suffix in commands is optional. "(block name)" is the name of the CL block to be unlinked. A source file can contain more than one block, so block names are not necessarily the same as the source file name, though they may be. "(tagname)" is the name of the bound data point that is configured in the point's NAME parameter. Letters on the command line can be uppercase or lowercase or a mixture of upper- and lowercase. (filename) cannot begin with "\$."

CL/MC programs are not linked nor loaded by the Link command. They are loaded from the Process Module Detail display in the Universal Station's Process Operator personality. See 3.2.2 in this publication and the *Process Operations Manual*.

5.3.1.2 Compile Command Options

The following options are available for the Compile command. The options are requested by keying in "-" followed by the option name for each option. More than one option can be included, and each is separated by a space. For example, a command line that requests several CL/MC options would have this form:

```
CL CALC2 -D -UL -NW -NL -NX -EX -OEP dev>vol -HE
```

Following is the full set of CL compile options (note that not all apply to CL/MC):

- **Debug**—Compile %DEBUG lines in the source file. See 3.3.3 in the *Control Language/Multifunction Controller Reference Manual*. Form:

```
-D (or -DEBUG)
```

- **Update Library**—Update the NIM library (see 3.2.1). Form:

```
-UL (or -UPDATELIB)
```

- **No Warnings**—Do not list notes and warnings in .LE or .LS files. Form:

```
-NW (or -NOWARN)
```

- **No Listing**—Do not generate a full listing file (.LS); list only error lines. Form:

```
-NL (or -NOLIST)
```

The -NL option automatically causes the -NX option to be selected; therefore, the CL/APM cross reference is omitted from the listing and both -NL and -NX are displayed as selected options at the bottom of the listing.

- **No Cross Reference**—Omit printing the CL cross reference (symbol table) at the end of the listing. Form:

`-NX` (or `-NOXREF`)

- **Extended C-Link Addressing**—Invokes the extended C-Link address calculation which allows the ability to communicate with up to 16 Multifunction Controllers on a single C-Link. Without this switch, addressing is based upon a maximum of eight MCs on a C-Link. See heading 4.9 in the *Control Language/Multifunction Controller Reference Manual* for details. Form:

`-EX` (or `-EXTEND`)

- **Override Equipment List Name**—Overrides the device name and volume name specified in an `%INCLUDE_EQUIPMENT_LIST` statement and/or the User Default Path. Form:

`-OEP device>vol` (or `-OVRWRTELP device>vol`)

Example: `-OEP NET>ELB`

- **Unit Instance Only**—Only the one specified unit instance of an included Equipment List will be compiled. Form:

`-UI unit_instance_name` (or `-UNITINST unit_instance_name`)

If no errors are detected during compilation of the unit instance selected,

- Listing (.LS) and object (.MO) files are generated only for that unit instance. Existing .LS and .MO files for that unit instance are overwritten.
- No error listing (.LE) file is generated for that unit instance and any existing .LE file for that unit instance is deleted.

If any error is detected during compilation of the unit instance selected,

- An error listing (.LE) file is generated that includes information only for that unit instance. If a .LE file for that unit instance already exists, it is overwritten.
- No listing (.LS) or object (.MO) file is generated, thus any existing .LS and .MO files for that unit instance are not replaced.

- **Optimize Object Generation**—(CL/PM and CL/APM only) Form:

`-OPT` (or `-OPTIMIZE`)

- **Heap Expansion**—Extends the amount of heap memory available for CL compilations if certain requirements are met. Form:

-HE (or -HEAPEXP)

CAUTION

The following requirements must be met to effectively use the Heap Expansion Option:

Heap Expansion Option Requirements:

R410 – R4XX

- Requires software release 410 or later 4XX releases,
- Universal Personality UNP,
- and a Universal Station with a minimum of 5 megawords of memory.

Without the heap expansion option, a Universal Station with Universal Personality and R410 software requires a minimum of 4 megawords of memory.

R5XX

- Requires software release UNP 5xx or later,
- and a minimum of 7 megawords of memory.

If the requirements in the caution box above are met, you can use the heap expansion switch (option) to provide more heap memory for CL compilations. This option can be especially useful in compiling large CL programs that use the Equipment List function.

CL programs that use Equipment List require more heap memory than programs that do not use Equipment List, and this reduces the number of blocks of object code that the CL Compiler can generate.

The following table shows the number of blocks of object code that the compiler can generate with and without the heap expansion option for programs that use Equipment List and programs that do not use Equipment List:

Controller	Number of Blocks of Object Code			
	With Heap Expansion Option (UP with 5 Mw Memory)		Without Heap Expansion Option (EP or UP)	
	With Equipment List	Without Equipment List	With Equipment List	Without Equipment List
APM	500 (16,000 words)	500 (16,000 words)	276 (8,832 words)	392 (12,544 words)
PM	336 (10,752 words)	336 (10,752 words)	223 (7,136 words)	256 (8,192 words)
MC/AMC	512 (8,192 words)	512 (8,192 words)	356 (5,696 words)	512 (8,192 words)

The maximum amount of heap memory available

- in the Engineering and Universal Personalities without expanded memory is 320,000 words,
- in the Universal Personality of R410 (with 5 Mw memory) is 700,000 words if the heap expansion option is used, and
- in the Universal Personality of R530 (with 7 Mw memory) is 1,400,000 words.

NOTE

To terminate a compilation in progress, press the combination of the "CTL" and "BREAK" keys on the engineering keyboard. The CL compiler recognizes the Break key during any of its three passes or during the listing generation.

5.3.1.3 Results of Compilation

The results of a successful compilation are reported both on the screen and in the listing (.LS) file (no files are modified on an unsuccessful compilation). In addition to reporting the status of custom data descriptors, the Compiler also reports the status of object files and of custom parameter list files.

The following are examples of the messages that report compilation status:

For a compilation that results in a new MC object file;

```
File 02050128.MO created
```

For a compilation that overwrites an existing MC object file;

```
File 02050128.MO overwritten
```

5.3.2 How to Invoke Compiler Commands

5.3.2.1 Compilation Precautions

Simultaneous CL/MC Compilations—An incorrect compilation may occur if two CL/MC compilations occur simultaneously (on different USSs), using the same MC library. Only one of the compilations will successfully modify the library; the other one may contain erroneous sequence/phase/step names or message character strings. To avoid this, verify that one such compilation is complete, before another is attempted.

5.3.2.2 Volume Handling and Pathnames

The Command Processor display doesn't provide prompters to request that required volumes be mounted in a disk/diskette drive for a CL compilation. If you don't make a needed volume available, an error message appears, indicating that the required volume or file could not be found. After you mount the required volume, retype your command line and press ENTER.

The pathnames used in executing Compiler commands are the default pathnames on the Utilities' Modify Default Volume Pathnames display.

The descriptions of typical CL data entry sessions in Section 3 provide examples of volume pathnames, and suggestions for handling disk/diskettes. Table 2-1 lists all volumes involved in CL data entry.

5.3.2.3 Invoking a Compile Command

1. If you are using some Engineering Personality activity other than the COMMAND PROCESSOR, hold CTL and press MENU to return to the Engineering Main Menu.
2. To invoke CL

Select the COMMAND PROCESSOR, type CL and press ENTER (Engineering Personality Overlay volume &OV2 must be available on a disk/diskette or an HM). The Control Language Compiler/Linker display appears.

3. The summary of CL commands and options appear as shown here.

```

26 Apr 91 15:22:55 4
COMMAND PROCESSOR  VER. 21.50  USER PATH : NET\QAS\
CL
Loading Overlay
Control Language Compiler/Linker V21.54 usage:
: CL filename (options)           - - Compile only
: CLK filename pointname (options) - - Compile/Link
: LK filename pointname           - - Link only
: UNLK blockname pointname        - - Unlink
options = (-d: -debug) (-nl: -nolist) (-nw: -nowarn)
          (-ul: -updatelib) (-nx: -noxref) (-od: -ovrwrtds)
          (-ex: -extend) (-upgap) (-etd: -exptindsp)
          (-opt: -optimize)
          (-oep dev>vol: -ovrwrtp dev>vol)
          (-ui instance_name: -unitinst instance_name)

```

2989

4. Check that your CL source file is available on the CL Source/Obj volume and that other volumes are available, as needed, for the type of structure you are compiling. To determine which are needed, refer to major steps and the sketches in Section 3.

Key in a Compile command line, and any options needed (see 5.3.1.2). Here we keyed in "c1 intreflx." "Intreflx" is the name of our source file.

5. Press ENTER. The Compiler makes several passes through the source file and each of these is noted on the display.

As each pass is processed, the line number of the current line being compiled is displayed on the Command Line to show progress of the compilation. Comment lines in the CL program are skipped and their line numbers are not displayed. During compiler processing of internal executive routines, a zero (0) line number is displayed. Here, the compiler made four passes and generated the listing in 17.4 seconds, and it detected no errors. The listing file (.LS) and the object file (.MO) are now available on the CL Source/Obj volume.

If errors are detected, they are indicated on a line like the one at the bottom of this display, the listing and object files are not created, and instead, an error file (.LE) is created on the CL Source/Obj volume. Section 6 provides guidelines for recovering from errors.

```

17 Aug 88 16:45:18 4
COMMAND PROCESSOR VER. 21.50 USER PATH : NET>MBD>

CL SAMPLE1 -NX -NW -UL
Loading Overlay
Elapsed time for Pass 1 = 3.6 seconds
Elapsed time for Pass 2 = 2.3 seconds
Elapsed time for Pass 3/MC = 1.1 seconds
Elapsed time for Listing = 8.6 seconds
Elapsed time for Compilation = 17.4 seconds
Existing PM object file:
File A0312921 .MO overwritten
Errors detected : None
Notes : 2
  
```

6338

```

05 May 87 14:31:03 2
COMMAND PROCESSOR VER. 7.4 USER PATH : NET>CL>

CL INTREFLX -UL
Elapsed time for pass 1 = 3.9 seconds
Elapsed time for compilation = 10.0 seconds
Errors detected : 1
  
```

3120

5.4 HOW TO USE COMMAND FILES

CL and Command Processor commands can be included in command files to cause them to be executed without human interaction. The command processor commands that facilitate the execution of command files are available through the Command Processor display.

DO (pathname)	Data Out
P (pathname)	Print
EC (pathname)	Execute Command File
PO (pathname)	Prompt Out
Pause	Pause (stop execution) until the operator presses ENTER.
End	End Command File Execution and this Utilities session.

where: (pathname) is a complete file pathname for either disk/diskette or HM.
(See Section 5.1 for a description of the Data Out and Print commands.)

The EC command directs the Universal Station to take commands from the specified file, rather than from the keyboard. The commands are displayed on the screen as they are read from the file. The responses to the commands also go to the screen, just as if they were being executed through interaction with an operator. Command execution continues until an End command is encountered or the end of the file is reached and you are returned to the Command Processor.

You can abort EC file execution by holding CTL and pressing COMND. When the command being executed is complete, execution stops, and you are returned to the Command Processor.

"&P" can be included in a command file to indicate comments that are not considered as commands. "&P" and the text that follows it appear on the screen as they are encountered, and if so specified by a PO command, they are also printed or appended to a file.

The PO command directs the US to copy the commands and responses as they appear on the screen to the specified printer, or to append them to the specified file. This continues until an End command or the end of the file terminates execution, or until a PO command with no pathname is executed.

The PAUSE command causes command execution to cease. Command execution continues when the ENTER button is pressed. The primary purpose of the PAUSE command is to allow you to mount a disk/diskette during the execution of an Execute file.

We recommend that you use a .EC suffix on your command files to identify their purpose. The EC and PROMPTOUT commands don't require this specific suffix, but you will find a .EC suffix a help in remembering the purpose of the file.

As an example, you could use the Text Editor to create a file named ECFILE in a volume named EC that looks like this:

```
PO NET>CL>SESSION.EC
&P This command file compiles two CL blocks
CL PUMP1

CL PUMP2
END
```

Then on the Control Language Compiler/Linker display, key in

```
EC NET>EC>ECFILE.EC
```

and press ENTER. The PO command directs the commands, the &P line, and all responses to file SESSION.EC. The two CL commands are then executed, and the End command terminates the session and returns you to the Command Processor.

If you wanted to review the commands and responses that were stored in SESSION.EC, you could key in

```
PR NET>CL>SESSION.EC
```

Then press ENTER. The content of the file appears on the display.

ERROR INDICATIONS AND RECOVERY

Section 6

Use parts 6.1 and 6.2 of this section to recover from errors detected by the Compiler commands. Part 6.3 provides information that will help you to recover from errors that are detected as a CL program is running in a Multifunction Controller.

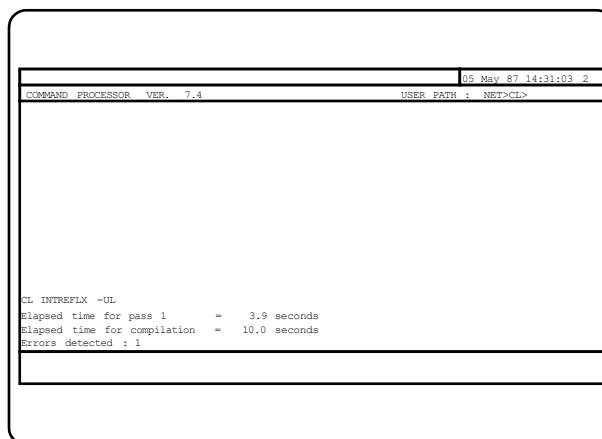
6.1 HOW ERRORS ARE INDICATED

CL data entry errors are indicated two ways:

- On the Command Processor Display.
- In error listing (.LE) files that the Compiler generates when it detects errors.

As each CL command is executed, messages appear on the Control Language Compiler/Linker display that indicate progress as the command is executed and that provide information that may or may not indicate errors.

The content of these messages must be considered, to determine if corrective action is needed or not. For example, this display indicates that a Compile command was executed; however, the last message on this display indicates that one error was detected during the compilation. That error can be found in the error listing (.LE) file.



3120

6.1.1 CL Compiler Error Messages

An alphabetical list of all of the error messages generated by the compile command is provided below. The information in the Source/Severity column has the following meaning:

- **Source**

CX—Communication and data access messages

P1, P2, P3MC—Compiler messages generated on pass 1, pass 2, or pass 3. These messages appear in the error listing (.LE) file.

The major activity in each of the Compiler passes is as follows:

Pass 1—CL syntax checking. Note: The compiler lists only the first syntax error it finds in each source code statement.

Pass 2—Parameter type checking

P3MC—Object code generated for a CL/MC compilation

- **Severity** (listed from most severe to least severe);

Fatal—An error that prevents further command execution or that prevents the CL Compiler from executing beyond the point where the error occurred

Severe—A severe, but not fatal error

Error—An error that did not prevent command execution

Note—The Compiler has done something that may be surprising or noteworthy

Warning—The user has done something noteworthy.

Messages	Source/Severity
%INCLUDE EQUIPMENT_LIST directive is not supported for CL/AM	P2-Fatal
%INCLUDE EQUIPMENT_LIST directive only allowed as the first line	P1-Error
%INCLUDE EQUIPMENT_LIST is not declared as the first line	P1-Error
&ASY volume not found	CX-Fatal
(filename) could not be created:<reason>	CX-Fatal
(filename) could not be deleted <reason>	CX-Fatal
(filename) could not be read: <reason>	CX-Fatal
(filename) Handler actually defined as (filename)	P2-Error
(filename) Not Found	P2-Error
(tagname.param) compile type does not match actual type	CX-Fatal
(tagname.param) does not exist	LK-Fatal
(tagname.param) is an array of point IDs that do not all refer to the same LCN	LK-Fatal
<name> cannot be redeclared	P2-Error
<name> expected	P1-Error
<name> is not a legal %RELAX option	P2-Error
<name> not found	P2-Error
<name> was not found in the system and -UL was not selected	CX-Error
A local variable is required here	P2-Error
A Loop may have only one REPEAT statement	P2-Error
A number cannot be divided by a time	P2-Error

(Continued)

Messages (Continued)**Source/Severity**

A remote variable is required here	P2-Error
ALARM time must be an integer constant (0-9999) for MC	P3MC/PM/APM-Error
Aliasing of Bound Data Point	P2-Error
Alias Name does not exist in Equipment List Object File	P1-Error
Allowed only before first Block in a Package or in Block data declarations	P2-Error
All enabled abnormal handlers are disabled	P2-Note
Argument multiply defined	P2-Error
Argument must be a Data Point Param of type Data Point	P2-Error
Argument must be a Data Point Param or a LOCAL	P2-Error
Argument must be an unsubscripted data point parameter	P2-Error
Argument must be of type ENUMERATION	P2-Error
Argument must be of type String Literal	P3APM-Error
Array expected	P2-Error
Array index must be of type number	P3MC/PM/APM-Error
Array too large	P3-Fatal
Arrays cannot be compared	P2-Error
Attempt to fetch element of off node array of strings: <pointname>.<parameter>	LK-Error
Attempt to redefine (name) as a parameter name	P2-Error
Attempt to redefine parameter (name) with different type	P2-Error
Attempt to store to (tagname.param) whose access lock is View Only	P2-Error
Attempted Aliasing	P2-Error
Attempted indirection	P2-Error
Attribute previously defined	P2-Error
Background Block Active on Bound Data Point—retry Link or Unlink	LK-Fatal
BACKGROUND is allowed only in Platform Subroutines and/or Functions	P2-Error
Block already exists at another Insertion Point	LK-Fatal
Block exceeds maximum number of prefetches allowed per block	LK-Fatal
Block insertion point must be BACKGRND when calling background routines	P3-Error
Block not on the Bound Data Point	LK-Fatal
Both KEEPENB and abnormal handlers not allowed in a Phase heading	P2-Error
Bound Data Point at maximum CLs	LK-Fatal
Bound Data Point has already been processed in a previous compilation	P2-Error
Bound Data Point is active	LK-Fatal
Bound Data Point not configured for CLs	LK-Fatal
C-Link data inconsistent in READ/WRITE: may indicate that no C-Link exists	P3MC-Error
Cannot create .GD file, FMS error: <reason>	CX-Fatal
Cannot declare a data type with REFERENCE_N	P2-Error
Cannot fetch values from an OUT argument	P2-Error
Cannot access this parameter from this sequence program	P3PM/APM-Error
Cannot generate code for this point reference	P3PM/APM-Error
Cannot get default pathname	CX-Error

(Continued)

Messages (Continued)**Source/Severity**

Cannot have LOCAL data definitions with the data type of Entity ID in a CL Block	P2-Error
Cannot INITIATE non Process Module Point	P3MC/PM/APM-Error
Cannot INITIATE own normal sequence	P3MC/PM/APM-Error
Cannot Link to a Library subroutine	LK-Fatal
Cannot loop in Phase Heading	P3MC/PM/APM-Error
Cannot loop in Step Heading	P3MC/PM/APM-Error
Cannot modify a local constant or enumeration literal	P2-Error
Cannot modify a Point ID—use Data Entity Builder	P2-Error
Cannot modify an expression	P2-Error
Cannot modify an IN argument	P2-Error
Cannot modify function	P2-Error
Cannot modify read-only parameter	P3MC/PM/APM-Error
Cannot multiply time by time	P2-Error
Cannot pass a member of an array of data Points	P2-Error
Cannot Read Pointer from Global memory	CX-Fatal
Cannot Read/Write subroutine argument	P3MC/PM/APM-Error
Cannot Read/Write this parameter	P3MC/PM/APM-Error
Cannot Reference @1.@2	LK-Fatal
Cannot reference data points with names greater than 8 characters across LCNs	P2-Error
Cannot reference Off-LCN \$AMIF Point	P2-Error
Cannot reference off-LCN points in a sequence program	P2-Error
Cannot Restart in emergency handler	P3MC/PM/APM-Error
Cannot SEND Enumeration types with blank state names	P3MC/PM/APM-Error
Cannot SEND Enumerations	P2-Error
Cannot SEND mode type value	P3MC-Error
Cannot send Arrays	P2-Error
Cannot send Conditions	P2-Error
Cannot store to an IOL parameter in a READ statement	P3PM-Error
Cannot use a parameter which requires a Prefetch in a CALL statement	P3PM-Error
Cannot use an entire array	P3MC/PM/APM-Error
Cannot use WHEN ERROR clause when INITIATE own handler	P3MC/PM/APM-Error
Checksum Error on Linker/Loader communication	LK-Fatal
Code size exceeds maximum of 12288 words	P3APM-Error
Compiler memory overflow! Program too large or statement too complex	CX-Fatal
Composite Data Point must reside in a MC	P3MC/PM/APM-Error
Constant is too big	P3MC/PM/APM-Error
Corrupted Data Base	LK-Fatal
Code Size exceeds maximum of 8192 words	P3PM-Fatal
Could not finish reading file (filename) <reason>	CX-Fatal
Could not get file attribute (filename) <reason>	CX-Fatal
Could not get memory in AM	LK-Fatal
Could not go to End of File (filename) <reason>	CX-Fatal
Could not open %Include_Set file @1: @2	CX-Error
Could not open file (filename) <reason>	CX-Fatal
Could not write to file (filename):<reason>	CX-Fatal
Custom Set @1 is not loaded in the AM	LK-Error

(Continued)

Messages (Continued)**Source/Severity**

DA error adding name to global directory, Build status=(value)	CX-Fatal
DA error converting entity name, minor_stat: (value)	CX-Error
DA error getting enumeration definition	CX-Fatal
DA error getting Point Parameters, se_stat: (value)	CX-Error
DA ext/int conv error, <reason>	CX-Fatal
DA int/ext conversion error, codes=(values)	CX-Fatal
Data Access Disk I/O Error:<reason>	CX-Fatal
Data Access error: Enumeration Set not found	CX-Error
Data Point @1 does not exist. Rebuild Equipment List.	P2-Fatal
Data Point @1 does not exist	LK-Fatal and Error
Data Point @1 not found—all nodes not operational	LK-Fatal and Error
Data Point Identifier expected	P2-Error
Data Point is too large	LK-Fatal
Data Point must be a MC point	P3MC-Error
Data Point must be in different MC from Bound Data Point	P3MC-Error
Data Point must be in the same UCN node as the bound data point	P3PM/APM-Error
Data Point must be in the same MC as Bound Data Point	P3MC-Error
Data Point must be on same Hiway as Bound Data Point	P3MC-Error
Data type DATA_POINT_ID must be followed by REFERENCE	P2-Error
Data Type name may not appear here	P2-Error
Data Type was not found in system	P2-Error
Declared size exceeds actual array bounds	P2-Error
Declared with invalid Data Type	P2-Error
Default value exceeds Port Size	P3CDS-Warning
Defined but not referenced	P2-Warning
Delaying for busy Loader	LK-Note
Destination must be a Data Point with a Mailbox	P2-Error
Different version of object exists in Unit <number>	
Usecount = <number>	LK-Fatal
Disk/Memory revision mismatch on Global Name File	CX-Fatal
Does not match Heading Name	P2-Error
Duplicate Declaration	P2-Error
ELSE cannot begin a Statement	P2-Error
Empty Custom Data Segment not permitted	P2-Error
Empty Object File	LK-Fatal
Empty SEND statements are not permitted in sequence programs	P3MC/PM/APM-Error
End of File expected	P1-Error
End of Line expected	P1-Severe
Entity Name could not be converted	CX-Error
Enumeration <name> is not valid on the UCN	CX-Error
Enumeration (name) not found—all nodes not operational	LK-Fatal
Enumeration literal expected	P2-Error
Enumeration literal may not appear here	P2-Error
Enumeration literal not implemented	P3MC-Error
Enumeration literal not in this Type	P2-Error
Enumeration name conflicts with Parameter List Name in Custom Library	CX-Error
Enumeration Set does not match actual enumeration set	CX-Error
Enumeration set <name> was not found in custom name library	P3CDS-Error
Enumeration state <name> not found - all nodes not operational	LK-Fatal

(Continued)

Messages (Continued)**Source/Severity**

Enumeration state not valid for this sequence program	P3PM/APM-Error
Enumeration type of this object is invalid	P3MC-Error
Enumeration types with blank state names are not supported	CX-Error
Equipment List Object version mismatch - Translate or rebuild Equipment List	CX-Error
Error attempting to get parameter information; S:@1 M:@2 PA:@3	CX-Fatal
Error during attempt to get parameter information; PA Status:<status>	CX-Fatal
Error during attempt to get SDE state names; Minor Status:<reason>	CX-Fatal
EU string truncated	P3CDS-Warning
Exceeded maximum of 3000 characters allowed for Value statements	P3CDS-Fatal
Expression is invalid in READ or WRITE statement	P3MC/PM/APM-Error
Expression is not constant or cannot be evaluated	P2-Error
Expression too complicated for evaluation	P3-Fatal
Fetching UCN Box Point parameters NN, FL, or TIME is not allowed in foreground CL	P3/LK-Error
File Manager Request Block—Allocation Error	CX-Fatal
File suffix is not Equipment List Object Suffix (QO)	P1-Error
Foreground fetch of Array Point <tagname>.TIME which has > 160 elements	LK-Error
Function Result must be scalar or string	P2-Error
Global Name File full	CX-Error
Handler already enabled for this condition	P2-Error
Handler contains multiple RESTARTS	P2-Error
Handler expected	P2-Error
Hardware location is required	P3MC/PM/APM-Error
Hardware location must be NN or FL	P3MC/PM-Error
Hardware location must be NN, FL, TIME, STR8, STR16, STR32, or STR64	P3APM-Error
Heading expected here	P1-Error
Heap memory requirements exceed available heap memory	CX-Fatal
I/O card type of bound MC is inconsistent	P3MC/PM/APM-Error
Identifier cannot be resolved in this context	P2-Error
Identifier expected here	P1-Error
Identifier length error	CX-Error
Identifier must be PM, APM, or MC—defaulted to PM	P2-Error
Identifier must be within 8 characters	P3MC-Error
Illegal character	P1-Error
Illegal compiler directive	P1-Severe
Illegal external Name	CX-Error
Illegal use of "%"	P1-Error
Incorrect format string	P3APM-Error
Index Type mismatch	P2-Error
INITIATE Point.Param not allowed in this sequence program	P2-Error
INITIATE SEQ not allowed in this sequence program	P2-Error
Insertion Point invalid for build type	LK-Error
Internal communication error (IDB) - Contact Honeywell TAC	CX-Fatal

(Continued)

Messages (Continued)**Source/Severity**

Internal error; Notify your Honeywell Representative:	
Converting LCN Name	LK-Fatal
Invalid access key	CX-Error
Invalid Array Base type	P2-Error
Invalid CALL argument	P3MC/PM/APM-Error
Invalid constant data type	P3MC/PM/APM-Error
Invalid Declaration	P3MC/PM/APM-Fatal
Invalid declaration for a Sequence Program	P2-Error
Invalid File Name	CX-Fatal
Invalid File Pathname or Pathname not specified	P1/CX-Error
Invalid in a Sequence	P2-Error
Invalid in Blocks	P2-Error
Invalid in this context	P2-Severe
Invalid %Include Set file name, must be 6 characters or less	P1-Error
Invalid Insertion Point (name)	LK-Error
Invalid insertion point for configured algorithm	CX-Error
Invalid mode enumeration for this point	P3MC/PM/APM-Error
Invalid Module Number, Entity does not exist	CX-Error
Invalid parameter on this point	P3MC/PM/APM-Error
Invalid point for this sequence	P2-Error
Invalid point type for this compilation	LK-Error
Invalid point type for this compilation	P2-Error
Invalid SEND destination	P3MC/PM/APM-Error
Invalid SEND item	P3MC/PM/APM-Error
Invalid SEND WAIT option for this Destination	P3MC/PM/APM-Error
Invalid Slot Number, Entity does not exist	CX-Error
Invalid State Name	P2-Error
Invalid subscript for this parameter	P3MC/PM/APM-Error
Invalid time expression	P3MC/PM/APM-Error
Invalid to indirect through an off-node point	LK-Fatal
Keylock error on attempt to store to (tagname.parameter)	LK-Error
Labels must be unique within a Program Block	P2-Error
Library full of Names/Strings, id not added	P3PM-Error
Library full of Sequence/Phase/Step id, id not added	P3MC/PM/APM-Error
Library full of Strings, String not added	P3MC/PM/APM-Error
Library Subroutine not permitted in a Package	P2-Error
Library Subroutine Object corrupted	LK-Fatal
Linking/Unlinking to points on other LCNs is not supported	LK-Fatal
Loader communication error	LK-Fatal
Loader still busy after one retry	LK-Fatal
Loader time out error	LK-Fatal
Loader/Data Access communication problem	LK-Fatal
Loader/Kernel communications problem	LK-Fatal
Local Subroutine cannot have a Bound Data Point	P2-Error
Local time not synchronized with network	CX-Note
Local variables may not be data point identifiers	P2-Error
Logical condition expected	P2-Error
Logical conditions cannot be compared	P2-Error
Logical or Enumeration type expected here	P2-Error
Loop Index not declared	P2-Error

(Continued)

Messages (Continued)**Source/Severity**

Loop was never repeated	P2-Warning
Lower bound must be less than upper bound	P2-Error
Malformed Identifier	P1-Error
Malformed number	P1-Error
Malformed Point Identifier	CX-Fatal
MAN, AUTO, CASC, and COM may be stored but not compared	P3MC-Error
Missing Abnormal Handler ID	P1-Error
Missing Closing Quote	P1-Error
Missing END	P1-Error
More than 12 IOL parameter fetches in one step	P3PM/APM-Error
MSF Error - Code:@1	P1/P2-Fatal
MSF FM Error - @1:@2	P1/P2-Fatal
MSF.LO is not installed in this node	P1-Fatal
Multiple locals assigned to this hardware location	P2-Error
Multiple stores of strings are not permitted	P2-Error
Must be a BDP Parameter	P2-Error
Must be a Parameter List	P2-Error
Must be a positive integer	P2-Error
Must be an integer	P2-Error
Must be APM resident point	CX-Error
Must be in the RESTART section of an ABNORMAL Handler	P2-Error
Must be MC resident point	CX-Error
Must be PM resident point	CX-Error
Must contain executable statements	P2-Error
Must link to point (name)	LK-Fatal
Name/String not found in Library and -UL was not selected	P3PM-Error
Name too long	P3-Error
Nested CALL invalid	P3MC/PM/APM-Error
Nested PACKAGE heading ignored	P2-Severe
Network in the EL Object does not match with compilation target	P2-Fatal
No available entries in Library for Sequence names, Sequence name not added	P3PM/APM-Error
No corresponding item for this Variable	P2-Error
No response from the remote AM node	LK-Fatal
Not a parameter	P2-Error
Not a Process Module point	CX-Error
Not a Subroutine	P2-Error
Not Enough Contiguous Memory Error	LK-Fatal
Not enough memory	CX-Fatal
Not supported	All-Error
Not supported on target machine	P3-Fatal
Not valid for AM	P2-Error
Null point ID only permitted in point ID arrays	LK-Fatal
Number of values differs from number of array elements	P2-Error
Numeric literal expected here	P1-Error
Object code memory requirement exceeds maximum allowable object size	LK-Fatal
Object corrupted in transmission from US to AM	LK-Fatal
Object File Access Error	LK-Fatal
Object File Corrupted	LK-Fatal

(Continued)

Messages (Continued)**Source/Severity**

Object file FMRB allocation error	LK-Fatal
Obsolete syntax - this name is ignored	P2-Error
Old File (filename) cannot be safely renamed (filename)	CX-Fatal
Only one %INCLUDE_EQUIPMENT_LIST directive is allowed	CX-Error
Only one Custom Data definition is permitted	P2-Error
Only valid in a Custom Data Segment	P2-Error
Parameter (name) not found—all nodes not operational	LK-Fatal
Parameter (name) not on this Data Point	P2-Error
Parameter conflict	P2-Error
Parameter Declaration expected	P2-Severe
Parameter Description truncated	P3CDS-Warning
Parameter List <name> was not found in custom name library	P3CDS-Error
Parameter List File (filename) is empty	P2-Error
Parameter List name conflicts with Enumeration Set Name in Custom Library	CX-Error
Parameter name conflicts with Segment name in Custom Library	CX-Error
Parameter name not allowed in this context	P2-Error
Parameter references used in a subscript must be On-Node	P3APM-Error
Parameters data type not supported	P2-Error
Passing enumeration literals as arguments in a CALL is not allowed	P3MC-Error
Phase Heading contains multiple ALARM clauses	P2-Error
Point ID change only valid after unlink and link	P3-Warning
Point Identifiers cannot be compared	P2-Error
Point is not NIM resident	CX-Error
Point must be declared in an EXTERNAL statement prior to the LOCAL declaration	P2-Error
Point must be on the same UCN as the Bound Data Point	CX-Error
Point must be resident on the UCN	CX-Error
PPS parameter not logical	P2-Fatal
Program is too large	P3MC/PM/APM-Error
PRTFL parameter of composite point contains only the status of the input slot	P3MC-Warning
Quoted string is too long. Maximum length allowed is <value>	P1-Error
Reference List entry created for each element of this array	P3-Note
References an off-LCN point; must link to IPP or Backgrnd Insertion Point	LK-Fatal
Remote variable in READ/WRITE must be in different box from Bound Data Point	P3MC/PM/APM-Error
Remote variable in READ/WRITE/INITIATE must be on same C-LINK as Bound Data Pt	P3MC-Error
RESUME must have PHASE destination	P3MC/PM/APM-Error
Routine must begin with step	P2-Error
Routine Out of Order	P2-Error
Same object exists with different access key in Unit <name>. Use count = <value>	LK-Fatal
Segment name conflicts with standard parameter or param name in Custom Library	CX-Error
Segment size exceeds maximum by <number> bytes	P3CDS-Error
Sequence must begin with a PHASE	P2-Error
Sequence/Phase/Step ID must be defined in Library	P3MC/PM/APM-Error

(Continued)

Messages (Continued)**Source/Severity**

Source File contains no code	P1-Fatal
Source File is empty	CX-Fatal
Statements not valid in Custom Data Segments	P2-Error
String exceeded maximum allowable length and was shortened to <value> characters	P3CDS-Warning
String is valid only in SEND statement	P3MC/PM-Error
String literal expected here	P1-Error
String literal type can only be general (G), integer (I), or real (R) format	P3APM-Error
String must be defined in Library	P3MC/PM/APM-Error
String must be within 8 characters	P3MC/PM/APM-Error
Subroutine/Function name must be less than or equal to 80 characters	CX-Error
Subscript must be an integer constant	P3MC/PM/APM-Error
Subscript outside array bounds	P3-Error
Subscript used too deep in an indirect reference	P3-Error
Syntax Error	P1-Severe
System Name conflict	CX-Fatal
Tagname access references only the upper subslot of this dual digital parameter	P3MC-Warning
Target does not define a loop	P2-Error
Temp File (filename) could not be renamed:(filename)	CX-Fatal
The AM must be configured with at least one BACKGROUND task	LK-Error
The argument being passed in is not a Data Point Parameter	P2-Error
The argument being passed in is not an Enumeration type	P2-Error
The Data Point Name @1 is invalid	LK-Fatal
The loop counter must be a LOCAL variable	P2-Error
The referenced parameter list (filename) was not found	P2-Error
The target of a CALL statement must be a subroutine	P2-Error
There are too many parameter references in this block	LK-Fatal
This array limited to one dimension	P2-Error
This data type may only be compared equal/unequal	P2-Error
This Include Set file has an error on line @1, column @2	CX-Error
This is not an array	P2-Error
This LOCAL variable is mapped to a Serial Interface IOP parameter	P3APM-Warning
This parameter name cannot be used in a Custom Data Segment	P3CDS-Error
Time expression may not be followed by a WHEN clause	P2-Error
Time may be wrong	CX-Note
Too few arguments	P2-Error
Too many arguments	P2-Severe
Too many Array elements	P3CDS-Fatal
Too many array elements individually initialized with the VALUE statement	P3CDS-Error
Too many Build Visible Parameters	P3CDS-Fatal
Too many constants in program	P3MC/PM/APM-Error
Too many enumerations	P2-Error
Too many Links to Unit	LK-Fatal
Too many Parameters in Segment, Entity Builder Limit	P3CDS-Error
Too many Segments in Package, Entity Builder Limit	P3CDS-Error

(Continued)

Messages (Continued)**Source/Severity**

Too many SEND items	P3MC/PM/APM-Error
Too many statements in a Step/Routine	P3MC/PM/APM-Error
Too many variables in this statement	P3MC/PM/APM-Error
Total Variables exceeds available memory	P3-Fatal
Type (name) expected	P2-Error
Type mismatch between variable type and hardware location	P3MC-Error
Type mismatch	P2-Error
Undefined Enumeration	CX-Error
Undefined Link Error	LK-Fatal
Undefined Symbol	P2-Error
Unexpected Attribute Statement	P2-Error
Unexpected End of File	P1-Error
Unexpected Record	LK-Fatal
Unexpected text after end of program	P1-Severe
Unit Instance name does not exist in Equipment List Object File	P2-Fatal
Unlink required—another block at specified Insertion Point	LK-Fatal
Unlink required—block already on Bound Data Point	LK-Fatal
Unmatched END statement ignored	P2-Severe
Unrecognizable Point Type	CX-Error
Unresolved forward reference	P2-Error
Use standard enumeration @1 instead of declaring enumeration states	P2-Error
Value cannot be Converted	P3-Fatal
Value does not match parameter type	P2-Error
Variables requiring prefetch are not permitted in the Phase alarm clause	P3APM-Error
Wrong Data Type for this Operator	P2-Error
Wrong number of Dimensions	P2-Error

6.2 CORRECTING CL SOURCE FILE ERRORS

The following are the steps used to correct an actual, but simple, CL source file error. A similar process can be used to correct all CL source file errors.

1. This is how the display appeared when compilation of the CL source file was completed. The last line on this display indicates that the Compiler detected one error. We need to look at the content of the error listing (.LE) file to find the error.

```

05 May 87 14:31:03 2
COMMAND PROCESSOR VER. 7.4      USER PATH : NET>CL>

CL INTREFIX -UL
Elapsed time for pass 1      =    3.9 seconds
Elapsed time for compilation =   10.0 seconds
Errors detected : 1
  
```

3120

2. We keyed in
PR \$f2>cl>intreflx.le
followed by ENTER, to request that the error listing be displayed.

```

05 May 87 14:31:03 2
COMMAND PROCESSOR VER. 7.4      USER PATH : NET>CL>

pr $f2>cl>intreflx.le
  
```

3121

3. The entire .LE file is displayed, line-by-line. When the end of the file is reached, a "Print Complete" message appears. You can now use PAGE FWD and PAGE BACK to page through the file to find the error messages.

```

05 May 87 14:31:03.2
COMMAND PROCESSOR VER. 7.4 USER PATH : NET>CL>
47 SET x = reflux.pv/feed.pv * (1.0 + (cp * delta.t/hv))
48 IF BADVAL(x) THEN GO TO badv
49 SET pvcalc = x
50 SET pvautost = normal
CL V14.2 INTERFLX 14 10 85 14:25
:023646 Page 2
51 --
52 IF init + off THEN EXIT
53 ESLE (SET CTRLINIT = ON:
54 & EXIT)
55 --
56 badpv : CALL SET BAD(pvcalc)
57 SET pvautost = bad
58 END Reflxtl
59
60 End Intreflx
61
***** Number of errors detected: 1
Print Complete

```

3123

4. We found our error on the first page of the .LE file. The **ERROR ** Missing Closing Quote message is just below the line with the error. The "^" is pointing to the beginning of the field with the error. The Compiler found that there was only one quote mark on the EU "POINT" line. It couldn't determine where the second or closing quote mark should be. We need to add a quote mark after the T in POINT.

```

05 May 87 14:31:03.2
COMMAND PROCESSOR VER. 7.4 USER PATH : NET>CL>
2
3 ENUMERATION status = Initial/good/bad
4
5 PARAM LIST p11
6 PARAMETER pvstatus : status --User defined status
7 PARAMETER pv
8 END p11
9
10 CUSTOM REFLXINT (Class General: Access Engineer)
11 PARAMETER Reflux: $RegCtl --Reflux Flow Controller
12 EU "POINT
^
**ERROR ** MISSING CLOSING QUOTE
13 PARAMETER FEED : $REG.CTL --FEED FLOW RATE
14 EU "POINT"
15 PARAMETER TVL : p11 --Vapor line Temperature
16 EU "POINT"
17 PARAMETER Trfx : p11 --Reflux Temperature

```

3124

5. Type ED followed by the pathname of the source file that needs to be corrected, example: \$f2>cl>intreflx.CL, and then press ENTER (be sure that the .CL file is available in the floppy drive or on an HM).
6. The first 23 lines of the .CL file appear. You can page forward and backward through this display. Our error is about halfway down the first page on the EU POINT line. We keyed in a quote mark after the word POINT to correct the error. Then we pressed LF, followed by E, and RETURN, to transfer the corrected file to the volume it came from.

```

TEXT EDITOR - RELEASE 3.3                                     01 Jan 83 00 18 01
1
PACKAGE INTREFLX
ENUMERATION status = Initial/good/bad
PARAM_LIST p11
PARAMETER pstatus : status      --user defined enumeration
PARAMETER pv
END p11
CUSTOM REFLXINT (Class General: Access Engineer)
PARAMETER Reflux: $Reg_Ctl      -- "Reflux Flow Controller"
EU "POINT"
PARAMETER FEED : $Reg_Ctl      -- "Feed Flow Rate"
EU "POINT"
PARAMETER TVL : p11           -- "Vapor Line Temperature"
EU "POINT"
PARAMETER Trflx : p11         -- "Reflux Temperature"
EU "POINT"

```

3125

7. The display returns to the Command Processor display.
8. Type cl intreflx, press ENTER to recompile the CL source file. This display shows that this time the compilation took 39.1 seconds, and there are now no errors.

```

COMMAND PROCESSOR VER. 7.4                                     05 May 87 14:31:03 2
USER PATH : NET>CL>
CL INTREFLX
Time is not synchronized
Elapsed Time for Pass 1      = 3.1 seconds
Elapsed Time for Pass 2      = 4.5 seconds
Elapsed Time for Pass 3/AM    = 6.9 seconds
Elapsed Time for Pass 3/CIS   = 8.3 seconds
Elapsed Time for Compilation = 39.1 seconds
Errors detected : None

```

3086

In the example of steps 1 through 7, an error was detected on the first Compiler pass, and passes 2 and 3 did not take place. For each pass to complete, the preceding pass must be error free. Once you have corrected an error in an early pass, you may still have more errors to correct in subsequent passes.

Most errors detected by the Link command are fatal; therefore, if you correct one such error, there may still be others to correct after you attempt to link again.

6.3 HOW TO DETERMINE THE LOCATION OF A CL/MC RUNTIME ERROR

The CL listing (.LS) file includes program-code location information. This information is in the LOC column in the examples provided below.

For CL/MC programs, the number that appears in the LOC column for each executable statement is the number that will appear in the statement field of the Process Module Detail Display for the bound data point, when the statement is executed.

The syntax of the IF statement can result in "hidden" statements for which no LOC number can be displayed. The statements in the consequent of IF statements can be written on the same line as the rest of the IF statement. This results in more than one executable statement for each CL/MC source code statement. In this case, the number in the LOC column is the number for the IF statement. The numbers for the statements in the consequent are not displayed. If it is important to know the LOC numbers for such statements, we suggest that each IF statement be written on a separate line, using the continuation character in column one. Here is a program that has two versions of the same IF statement, one containing hidden statements and the other with no hidden statements.

Line	Loc	Text
1		SEQUENCE locnum(point hg05pm01)
2		PHASE ph01
3		
4		--EXAMPLE OF AN IF statement with hidden statements
5	1	IF f1(01) THEN (SEND : "W"; SET f1(02) = OFF)
6	4	ELSE IF f1(02) THEN SEND : "X"
7	6	ELSE (SEND : "Y"; SEND : "Z")
8		
9		--The same IF statement with no hidden statements
10	8	IF f1(01) THEN
11	9	& (SEND : "W";
12	10	& SET f1(02) = OFF)
13	11	ELSE IF f1(02) THEN
14	12	& SEND : "X"
15	13	ELSE (SEND : "Y";
15	14	& SEND : "W")
17		
18		END locnum

Index

Topic	Section Heading
CL Compiler/Linker Commands	5.3
CL Data Entry Error Messages	6.1.1
CL Data Entry Key Functions	4.1
CL Data Entry Sessions, Typical	3
CL Source (.CL) File, How to Create	5.2.1
CL Source File Errors, Correcting	6.2
CL/MC Sequence Program, Installing	3.2
Command Files, How to Use	5.4
Command Options	5.3.1.2
Commands	5
Commands	5.3.1.1
Compilation Precautions	5.3.2.1
Compile Command, Invoking	5.3.2.3
Compiler/Linker Command Summary	5.3.1
Compiler/Linker Commands	5.3
Compiler/Linker Commands, How to Invoke	5.3.2
Copy All or Part of a Source File	5.2.2
Correcting CL Source File Errors	6.2
Create a CL Source (.CL) File	5.2.1
Data Entry Publications	1.3
Data Entry Sessions, Typical	3
EC Command	5.4
Engineer's Keyboard	4
Engineering Personality Running, Needed Volumes Available	2.3
&Enn Volume and CL/MC .MO File(s)	3.2.2
Error Location, How to Determine	6.3
Error Indications and Recovery	6
Error Messages	6.1.1
Errors, How Indicated	6.1
Execute Command	5.4
HG Message Libraries	3.2.1
How Errors are Indicated	6.1
How to Copy All or Part of a Source File	5.2.2
How to Create a CL Source (.CL) File	5.2.1
How to Determine Error Locations	6.3
How to Invoke Compiler/Linker Commands	5.3.2
How to Use Command Files	5.4
How to Use the Typical Sessions Section	3.1
How to Use This Publication	1.2
Install CL/MC Sequence Program	3.2
Invoking a Compile Command	5.3.2.3
Key Functions, CL Data Entry	4.1
Keyboard, Engineer's	4
Message Libraries, HG	3.2.1
.MO Object File, Preparing to Load	3.2.2
Moving CL Applications	3.3
-OCD Option	5.3.1.2
Options, Commands	5.3.1.2
Overwrite Custom GDFs (-OCD)	5.3.1.2
Pathnames and Volume Handling	5.3.2.2

Index

Topic	Section Heading
Precautions for compilations	5.3.2.1
Preceding System Startup Tasks Completed	2.1
Prepare to Load the .MO Object File	3.2.2
Prerequisites, CL Data Entry Session	2
Prerequisites, CL/MC Compile and Load Session	2.4
PROMPTOUT Command	5.4
Publications, Data Entry	1.3
Publications, Reference	1.3
Publications, Related	1.3
Reference Publications	1.3
Related Publications	1.3
Sequence Program, CL/MC, Installing	3.2
Source File Errors, Correcting	6.2
Startup Tasks, Preceding	2.1
Text Editor Commands	5.2
Typical CL Data Entry Sessions	3
Universal Station and Volume Handling Guidelines	2.3.1
User Volumes Established with Sufficient Capacity	2.2
Using Text Editor Commands in CL Data Entry	5.2
Using Utility Commands in CL Data Entry	5.1
Using CL Applications on Another System	3.3
Utility Commands in CL Data Entry	5.1
Volume Handling Guidelines	2.3.1
Volume Handling and Pathnames	5.3.2.2
Volumes Needed	2.3

FAX Transmittal

FAX No.: (602) 313-4842

TO: Maria Nelson _____

Total FAX pages: _____
(including this page)

READER COMMENTS

Title of Document: **CL/MC Data Entry**

Issue Date: **12/97**

Document Number: **PC11-585**

Comments: _____

Recommendations: _____

FROM:

Name: _____ **Date:** _____

Title: _____

Company: _____

Address: _____

City: _____ **State:** _____ **ZIP:** _____

Telephone: _____ **FAX:** _____

You may also call 800-822-7673 (available in the 48 contiguous states except Arizona; in Arizona dial 602-313-5558, or write to:

Honeywell Inc.
Industrial Automation and Control
Automation College
2820 West Kelton Lane
Phoenix, AZ 85023-3028



Industrial Automation and Control

Automation College

2820 W. Kelton Lane

Phoenix, AZ 85023-3028

Helping You Control Your World